

TP2

Programmation orientée objet : le village d'habitants

Classes, encapsulation, héritage et polymorphisme

Contraintes et Remise du TP

- Ce TP reprend et prolonge l'exemple du **village d'habitants** introduit en cours et lors du TP précédent : vous allez transformer les fonctions procédurales de ce village en un ensemble de classes.
- La correction tiendra compte de la brièveté des méthodes que vous écrivez (évités les méthodes de plus de 25 lignes) - n'hésitez pas à découper une méthode en plusieurs sous-fonctions plus courtes.
- Vos classes et vos méthodes doivent être documentées (docstring).
- Pensez à utiliser pylint pour avoir un code qui soit le plus qualitatif possible.
- Les noms de variables, de fonctions et de méthodes doivent être explicites et en minuscules (snake_case), les noms de classes en CamelCase.
- Ce TP est un travail **personnel** (pas de binôme) : chacun doit produire et rendre son propre code.
- Le rendu se fait exclusivement via votre **dépôt GitHub** personnel. Vous avez 4h pour faire le TP, et le commit pris en compte sera le dernier réalisé avant minuit.

Exercice 1: Rendu via GitHub (4 points)

Comme pour le TP précédent, ce TP se rend via votre dépôt GitHub TP_OOP.

1. 1 point Récupérez la dernière version de votre dépôt TP_OOP, et créez-y un nouveau dossier `tp2/` qui contiendra exclusivement les fichiers de ce TP.

```
cd TP_OOP
git pull
mkdir tp2
cd tp2
```

2. 2 points Dans le dossier `tp2/`, organisez votre code en un fichier par question (`habitant.py`, `village.py`, `personnes.py`, `tests.py`), et effectuez un commit **après chaque question terminée**, avec un message explicite décrivant ce qui a été fait.

```
git add habitant.py
git commit -m "Question 3 : classe Habitant avec constructeur"
git push
```

3. 1 point Avant la remise, vérifiez que l'historique de votre dépôt reflète bien votre progression au fil du TP (`git log --oneline`), et que votre dernier commit est bien poussé sur GitHub avant minuit.

Exercice 2: Environnement virtuel (2 points)

Vous réutilisez l'environnement conda `tp_oop` créé lors du TP précédent.

1. 1 point Activez votre environnement `tp_oop` et vérifiez que `pylint` y est bien installé.

```
conda activate tp_oop
pylint --version
```

2. 1 point Ce TP nécessite le paquet `multipledispatch` (question 6). Installez-le, puis régénérez `environment.yml` à la racine du dépôt et committez-le.

```
conda install conda-forge::multipledispatch
conda env export > environment.yml
git add environment.yml
git commit -m "Mise a jour environnement : ajout multipledispatch"
```

Exercice 3: De la fonction à la classe `Habitant` (4 points)

Au TP précédent, chaque habitant du village était représenté de façon éclatée : un tuple pour son adresse, une entrée dans un dictionnaire pour ses biens, etc. On va maintenant regrouper tout cela dans une classe `Habitant`.

1. 2 points Créez une classe `Habitant` avec un constructeur `__init__(self, nom, age, adresse, animaux=None)`, où `animaux` est un dictionnaire associant à chaque type d'animal (clé) une quantité (valeur), comme au TP précédent, par exemple `{"vaches": 3, "moutons": 5}`. Si `animaux` n'est pas fourni, initialisez un dictionnaire vide.
2. 2 points Ajoutez deux méthodes : `affichage_adresse(self)`, qui affiche une chaîne du type "Aldric habite a Rue A", et `compte_animal(self, animal)`, qui renvoie le nombre d'animaux du type donné possédés par l'habitant (0 si absent).

```
h1 = Habitant("Aldric", 25, "Rue A", {"vaches": 3})

assert h1.nom == "Aldric"
assert h1.compte_animal("vaches") == 3
assert h1.compte_animal("moutons") == 0
h1.affichage_adresse() # affiche "Aldric habite a Rue A"
```

Exercice 4: Encapsulation (4 points)

Un utilisateur mal intentionné pourrait directement écrire `h1.age = -5`, ce qui n'a pas de sens. Vous allez protéger les attributs de `Habitant`, comme vu en cours.

1. 2 points Rendez tous les attributs de `Habitant` privés (préfixe `__`), et ajoutez les accesseurs (`get_nom`, `get_age`, `get_adresse`, `get_animaux`) et mutateurs (`set_nom`, `set_age`, `set_adresse`, `set_animaux`) correspondants. Adaptez `affichage_adresse` et `compte_animal` en conséquence.
2. 2 points En utilisant les décorateurs `@property` et `@age.setter`, transformez `age` en propriété. Le `setter` doit lever une `ValueError` si la valeur fournie est négative ou supérieure à 130.

```
h1.age = 26
assert h1.age == 26

try:
    h1.age = -5
    assert False, "une ValueError aurait du etre levee"
except ValueError:
    pass
```

On reprend l'exemple vu en cours pour regrouper les habitants dans un village, en illustrant la différence entre composition et agrégation.

Exercice 5: Composition et agrégation : la classe `Village` (6 points)

1. 2 points

Créez une classe `Village` avec un constructeur `__init__(self, nom)` qui initialise le nom du village et une liste vide `habitants`.

2. 3 points Ajoutez deux méthodes qui permettent toutes deux d'ajouter un habitant à la liste, mais avec une sémantique différente :
 - `ajouter_habitant_composition(self, nom, age, adresse, animaux=None)` crée un nouvel objet `Habitant` et l'ajoute à la liste (le village est propriétaire de l'habitant qu'il vient de créer);
 - `ajouter_habitant_agregation(self, habitant)` ajoute à la liste un objet `Habitant` qui existe déjà (le village utilise un habitant, mais n'en est pas seul propriétaire : le même habitant peut appartenir à plusieurs villages).
 Ajoutez également `afficher_habitants(self)` qui affiche chaque habitant du village.

```
pytown = Village("PyTown")
pytown.ajouter_habitant_composition("Aldric", 25, "Rue A", {"vaches": 3})

elise = Habitant("Elise", 28, "Rue B", {"poules": 10})
pytown.ajouter_habitant_agregation(elise)

autre_village = Village("VillageVoisin")
autre_village.ajouter_habitant_agregation(elise)  # meme habitant dans 2 villages

assert len(pytown.get_habitants()) == 2
assert elise in autre_village.get_habitants()
```

3. 1 point En 2-3 phrases dans un commentaire de votre fichier `village.py`, expliquez, avec vos propres mots, pour quoi `ajouter_habitant_composition` illustre une relation de composition alors que `ajouter_habitant_agregation` illustre une relation d'agrégation.

Exercice 6: Bonus : surcharge du constructeur (2 points)

Comme vu en cours, la surcharge n'existe pas nativement en Python; on utilise le paquet `multipledispatch`.

1. 2 points En utilisant `@dispatch`, écrivez une fonction `set_info` surchargée pour la classe `Habitant`, avec au moins deux versions : une qui met à jour uniquement le nom à partir d'une chaîne de caractères (`@dispatch(object, str)`), et une qui met à jour le nom et l'âge à partir d'une chaîne et d'un entier (`@dispatch(object, str, int)`).

```
from multipledispatch import dispatch

h2 = Habitant("Bob", 40, "Rue C")
set_info(h2, "Robert")           # met a jour le nom seulement
set_info(h2, "Robert", 41)       # met a jour le nom et l'age
```

Tous les habitants ne se comportent pas de la même façon selon leur âge. On souhaite néanmoins garantir que **tout** habitant sait calculer le nombre d'années qu'il lui reste avant la retraite.

Exercice 7: Héritage : Adulte et Enfant (6 points)

1. 3 points

Transformez `Habitant` en classe abstraite (héritant de `ABC`, module `abc`), qui conserve les attributs et méthodes déjà écrits, et ajoute une méthode abstraite `calcul_nombre_annee_avant_retraite(self)` (`@abstractmethod`). Vérifiez qu'il est désormais impossible d'instancier `Habitant` directement.

2. 3 points Créez deux classes dérivées, qui appellent `super().__init__(...)` pour initialiser les attributs communs. Le constructeur de chacune vérifie que l'âge est cohérent, et `calcul_nombre_annee_avant_retraite(self)` est implémentée différemment dans chaque classe (âge de départ à la retraite fixé à 62 ans) :

```
# Adulte : leve une ValueError si age < 18
# calcul_nombre_annee_avant_retraite() renvoie :
#   - "Déjà a la retraite" si age >= 62
#   - 62 - age sinon

# Enfant : leve une ValueError si age >= 18
# calcul_nombre_annee_avant_retraite() renvoie toujours :
#   - "Erreur: un enfant ne peut pas calculer sa retraite"

adulte = Adulte("Dupont", "Marie", 35, "Rue A")
enfant = Enfant("Martin", "Lucas", 12, "Rue B")

assert isinstance(adulte, Habitant)
assert adulte.calcul_nombre_annee_avant_retraite() == 27
assert "enfant" in enfant.calcul_nombre_annee_avant_retraite()

try:
    Enfant("Oups", 25, "Rue C")
```

```

    assert False, "une ValueError aurait du etre levee"
except ValueError:
    pass

```

Exercice 8: Polymorphisme (5 points)

On veut maintenant afficher tout habitant du village de façon uniforme, sans se soucier de son type précis (Adulte ou Enfant).

1. 2 points Ajoutez à `Habitant` une méthode spéciale `__str__(self)` qui renvoie une chaîne du type "Marie Dupont, 35 ans, habite a Rue A". Vérifiez que `print(adulte)` affiche bien cette chaîne.
2. 2 points Écrivez une fonction `affichage(h: Habitant)` qui affiche un habitant en utilisant `str(h)`. Appelez-la sur un `Adulte` et sur un `Enfant` : la fonction ne doit contenir **aucun** `if/instance` pour distinguer les deux types — c'est le polymorphisme qui doit faire le travail.

```

affichage(adulte)
affichage(enfant)
# Chaque habitant s'affiche correctement, quel que soit son type reel

```

3. 1 point En une phrase, expliquez pourquoi le fait d'avoir rendu `calcul_nombre_annee_avant_retraite` abstraite dans `Habitant` (question 6) plutôt que de simplement la définir avec `pass` renforce la sûreté du polymorphisme utilisé dans `affichage`.

Exercice 9: Tests unitaires (7 points)

Vous allez tester les briques principales de ce TP à l'aide du module `unittest`, en couvrant un cas usuel et un cas limite par fonction/méthode testée.

```

import unittest

class TestHabitant(unittest.TestCase):
    """Tests pour la classe Habitant et l'encapsulation."""

    def test_age_setter_valide(self):
        ...

    def test_age_setter_invalide(self):
        """Cas limite : age negatif."""
        ...

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

1. 3 points Écrivez la classe `TestHabitant` ci-dessus dans son intégralité (au minimum : `compte_animal` avec un cas limite où l'animal n'est pas possédé, et le `setter` de `age` avec une valeur invalide).
2. 2 points Écrivez une classe `TestVillage` testant les deux méthodes d'ajout d'habitant de la question 5 (composition et agrégation). Le cas limite attendu est un même habitant ajouté par agrégation à deux villages différents : il doit apparaître dans les deux listes.
3. 2 points Écrivez une classe `TestHeritage` qui vérifie que `calcul_nombre_annee_avant_retraite()` renvoie bien un résultat cohérent pour un `Adulte` et pour un `Enfant`, et que la création d'un `Enfant` de 20 ans lève bien une `ValueError` (cas limite).